Adaptive RDMA Optimization for Distributed LLM Workloads

Update 12: Work-First Scheduling for Tensor Transport

From Cilk's Work-First Principle to RDMA-Aware Task Scheduling

Nathan Gopee
MS Computer Science, SUNY New Paltz
Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

February 2026

Abstract

This update examines the Work-First Algorithm (WFA) from its origins in Cilk's work-stealing scheduler through its implications for CPU scheduling, memory hierarchy management, and distributed tensor transport over RDMA. The work-first principle—minimize overhead on the serial execution path at the expense of the parallel coordination path—provides the theoretical grounding for the WFA tensor classifier introduced in Update 10. We connect Frigo, Leiserson, and Randall's continuation-stealing strategy to Ryan Fleury's lane-based multi-core model, analyze the memory scheduling consequences (cache locality, NUMA awareness, space bounds), and show how these ideas map onto RDMA transport decisions for distributed LLM inference.

Contents

1 The Work-First Principle **3**
1.1 Two-Clone Compilation . . . 3
1.2 Continuation Stealing vs. Child Stealing . . . 3

2 CPU Scheduling: Work-Stealing Mechanics **4**
2.1 The Blumofe-Leiserson Theorem . . . 4
2.2 Work-First vs. Help-First in Practice . . . 4

3 Memory Scheduling: Cache and NUMA Implications **4**
3.1 Cache Miss Analysis . . . 4
3.2 NUMA-Aware Work-Stealing . . . 5
3.3 Space Bounds and Stack Management . . . 5

4 Fleury's Lane Model: Multi-Core by Default **5**
4.1 Connection to Work-First Scheduling . . . 6
4.2 Memory Implications of the Lane Model . . . 6

5 Mapping to RDMA Tensor Transport **6**
5.1 The Tensor Transport Analogy . . . 6
5.2 WFA Classifier as Scheduling Policy . . . 7
5.3 Continuation Stealing over RDMA . . . 7
5.4 Cache Hierarchy → Memory Hierarchy → Network Hierarchy . . . 7

6 Implications for the Middleware **8**
6.1 Scheduling Policy Selection . . . 8
6.2 NUMA-Aware Buffer Placement . . . 8
6.3 Space-Bounded Transfer Queues . . . 8

7 Quantitative Analysis **9**
7.1 Steal Frequency in Tensor Transport . . . 9

8 Connection to Prior Updates **9**

9 Conclusions **10**

1 The Work-First Principle

The work-first principle originates from Frigo, Leiserson, and Randall's implementation of Cilk-5 [4]:

> *Minimize the scheduling overhead borne by the work of a computation. Concentrate on reducing overheads that contribute to T_1, even at the expense of overheads that contribute to T_∞.*

The insight is counterintuitive. In a computation with work T_1 (total operations) and span T_∞ (critical path length), the expected runtime on P processors under work-stealing is:

$$E[T_P] = \frac{T_1}{P} + O(T_\infty) \tag{1}$$

The T_1/P term dominates for well-parallelized programs. Shaving overhead from the common serial execution path yields greater returns than reducing the rare parallel-coordination overhead.

1.1 Two-Clone Compilation

Cilk-5 implements this via a two-clone compiler strategy [4]:

- **Fast clone**: Nearly identical to the serial C version. Executes in the common case where no stealing occurs. Spawn overhead is 2–6× the cost of a C function call.
- **Slow clone**: Contains full parallel bookkeeping (deque management, frame setup). Executes only when work is actually stolen by another processor.

The runtime uses a Dijkstra-like mutual-exclusion protocol for the ready deque that avoids locks in the fast-clone path. The result: programs pay nearly zero overhead when running on a single core, and the scheduling machinery only activates on the rare steal path.

1.2 Continuation Stealing vs. Child Stealing

The work-first principle mandates **continuation stealing**. On a `spawn`, the spawning worker immediately executes the child (the "work") and pushes the continuation onto the deque for potential stealing. This contrasts with child stealing, where the child is enqueued and the parent continues.

Table 1: Continuation stealing (work-first) vs. child stealing comparison

Property	**Continuation Stealing**	**Child Stealing**
On spawn	Execute child, enqueue continuation	Enqueue child, continue parent
Execution order	Depth-first (mimics serial)	Breadth-first divergence
Space bound	$O(S_1 \cdot P)$ – provably bounded	Unbounded in worst case
Cache behavior	Preserves serial access pattern	Departs from serial locality
Implementation	Requires compiler support	Works as a library
Steal overhead	Higher (continuation is heavier)	Lower (child is lighter)

The space bound is critical: continuation stealing guarantees that P processors use at most P times the single-processor stack space [2]. Child stealing provides no such guarantee—fork-heavy loops can grow work queues without bound.

2 CPU Scheduling: Work-Stealing Mechanics

2.1 The Blumofe-Leiserson Theorem

Blumofe and Leiserson [2] proved that randomized work-stealing executes a fully strict computation in expected time satisfying Equation 1. This is optimal to within a constant factor of the lower bound $\max(T_1/P, T_\infty)$.

Each processor maintains a double-ended queue (deque) of ready tasks:

1. A worker pushes and pops from the **bottom** of its own deque (depth-first, following serial order).
2. When idle, a worker becomes a **thief** and steals from the **top** of a randomly chosen victim's deque (oldest/shallowest tasks).
3. Stealing the shallowest task maximizes the amount of work obtained per steal, minimizing total steal count.

The expected number of steals is $O(P \cdot T_\infty)$—proportional to the critical path, not the total work. Since $T_\infty \ll T_1$ for well-parallelized programs, steals are rare events. This is why the work-first principle pays off: the fast path (no steal) runs almost all the time.

2.2 Work-First vs. Help-First in Practice

Guo et al. [5] formalized the distinction between work-first and help-first (breadth-first) policies for async-finish task parallelism. Their key finding: neither policy universally dominates.

- **Work-first wins** for deep recursion with good temporal locality (e.g., divide-and-conquer, tree traversals).
- **Help-first wins** for broad fork-join with many short-lived tasks where the continuation is cheaper than the child.

This motivated SLAW [6], which selects the policy per-task at runtime based on locality hints—an adaptive approach that directly parallels our WFA tensor classifier selecting transport strategies per-tensor based on access patterns.

3 Memory Scheduling: Cache and NUMA Implications

3.1 Cache Miss Analysis

Acar, Blelloch, and Blumofe [1] analyzed the parallel cache complexity under work-stealing:

$$Q_P \leq Q_1 + O\left(\frac{P \cdot T_\infty \cdot M}{B}\right) \tag{2}$$

where Q_1 is the serial cache complexity, M is cache size, and B is cache line size. The additive term represents cache misses caused by steal operations disrupting locality—each steal can flush up to M/B cache lines.

The work-first principle minimizes this penalty because:

1. **Depth-first execution preserves serial locality**. A worker follows the same memory access pattern as the serial program until a steal occurs.

2. **Steals are rare** ($O(P \cdot T_\infty)$ total). Each steal incurs a bounded cache disruption, but there are few of them.

3. **Stolen tasks are large**. Stealing the shallowest continuation yields the most work, amortizing the cache refill cost over more computation.

However, the bound can be tight in adversarial cases. Acar et al. showed that a computation with $O(n)$ work and $O(1)$ cache misses on one processor can incur $\Omega(n)$ cache misses on just two processors when a steal destroys critical cache state.

3.2 NUMA-Aware Work-Stealing

Standard randomized work-stealing is **locality-oblivious**: a thief steals from a uniformly random victim regardless of NUMA topology. On modern multi-socket systems with non-uniform memory access costs, this causes unnecessary cross-socket traffic.

SLAW [6] addresses this by grouping workers into "places" aligned with the NUMA topology. Workers preferentially steal from within their place (same socket, shared LLC) before attempting remote steals. The adaptive policy selection further reduces NUMA penalties: work-first policy keeps data close to the producer (good for intra-socket), while help-first distributes work across sockets when local queues are saturated.

3.3 Space Bounds and Stack Management

The $O(S_1 \cdot P)$ space bound under continuation stealing [2] has a direct memory scheduling consequence: each processor's stack grows at the same rate as the serial execution. No processor accumulates unbounded pending work.

Under child stealing, a parallel `for` loop spawning n iterations immediately enqueues n tasks, consuming $O(n)$ space regardless of P. This is why systems like Intel TBB and .NET's Task Parallel Library (which use child stealing) require work-limiting heuristics to avoid memory exhaustion—a scheduling concern that continuation stealing avoids by construction.

4 Fleury's Lane Model: Multi-Core by Default

Ryan Fleury [3] proposes inverting the traditional parallelism model: instead of writing single-threaded code with special multi-core carve-outs, start with all N cores executing the same entry function parameterized by thread index. Serial sections use `if(LaneIdx() == 0)`.

This "multi-core by default" model resembles GPU shader programming more than traditional CPU threading. The key primitives:

- `LaneRange(count)`: Uniformly distributes iteration ranges across threads, handling remainders.

- `LaneSync()`: Barrier synchronization between threads in a group.

- `LaneSyncU64()`: Broadcasts small values across threads for data sharing without persistent statics.

4.1 Connection to Work-First Scheduling

Fleury's model is implicitly work-first: every core begins with work (the full program), and the distribution primitives (`LaneRange`) partition work uniformly without a steal mechanism. This works because:

1. **Uniform work**: When tasks have similar cost, static partitioning matches work-stealing's steady state (where steals are rare because queues stay balanced).
2. **Variable work**: Fleury uses atomic counters for dynamic assignment—functionally equivalent to a centralized work queue, which converges to work-stealing behavior under contention.
3. **Few expensive tasks**: Requires algorithmic redesign (Fleury switched from comparison sort to radix sort). This parallels the work-first insight that reducing T_1 matters more than reducing coordination overhead.

The model trades the generality of work-stealing (handles arbitrary task graphs) for simplicity (SPMD execution with barriers). For LLM inference, where the computation graph is known at compile time and tasks within a layer are homogeneous, this trade-off favors the lane model.

4.2 Memory Implications of the Lane Model

The lane model has favorable memory scheduling properties:

- **No deque overhead**: No per-thread task queues to manage. Memory footprint is purely the thread stacks plus shared data.
- **Predictable access patterns**: Each thread's data partition is known statically, enabling NUMA-aware allocation.
- **Stack as thread-local storage**: Each thread's stack is available throughout execution, avoiding heap allocation for per-thread state.

These properties align with RDMA buffer management, where memory regions must be pre-registered with the NIC and pinned in physical memory. Predictable access patterns enable pre-registration of the exact memory regions each thread will touch.

5 Mapping to RDMA Tensor Transport

The work-first principle translates directly to distributed tensor transport decisions.

5.1 The Tensor Transport Analogy

In distributed LLM inference, the "work" is GPU computation on tensor data, and the "coordination overhead" is RDMA transfer between nodes. The work-first principle says: *minimize overhead on the computation-critical path, even at the expense of the transfer coordination path.*

Table 2: Work-first principle mapped to tensor transport

Cilk Concept	Tensor Transport Analog	Implementation
Work (T_1)	GPU compute on tensors	Kernel execution time
Span (T_∞)	Longest pipeline stage	Critical-path layer
Fast clone	HOT path: dedicated QP, polled	Zero-copy RDMA read
Slow clone	COLD path: batched, compressed	Sparse-encoded bulk transfer
Steal (rare event)	Cross-node transfer	RDMA send/recv
Depth-first execution	Compute before transfer	Overlap compute with prefetch
Deque (bottom = local)	Local GPU memory	No transfer needed
Deque (top = stealable)	Registered RDMA buffers	Available for remote read

5.2 WFA Classifier as Scheduling Policy

The WFA tensor classifier (Update 10) implements an adaptive scheduling policy analogous to SLAW [6]:

- **HOT tensors** (access_count $> \theta_2$): Dedicated queue pair, polled completions, pinned memory. This is the "fast clone"—minimal per-transfer overhead on the critical path.
- **WARM tensors** ($\theta_1 <$ access_count $\leq \theta_2$): Shared queue pair, event-driven completions. Moderate overhead, balancing resource usage.
- **COLD tensors** (access_count $\leq \theta_1$): Batched transfers, compression enabled. This is the "slow clone"—higher per-transfer overhead, but these tensors are off the critical path.

The classifier's idle timeout (t_{idle}) implements demotion: tensors that stop being accessed cool down from HOT to WARM to COLD, freeing dedicated resources. This mirrors how work-stealing reclaims idle processors—an idle worker becomes a thief rather than holding resources.

5.3 Continuation Stealing over RDMA

Shiina and Taura [7] demonstrated distributed continuation stealing over MPI-3 RMA (one-sided RDMA operations). Their findings are directly relevant:

1. **Continuation stealing outperformed child stealing** in distributed settings. The depth-first locality advantage of continuation stealing is amplified by the high cost of remote memory access over RDMA—each "cache miss" is now a network round trip.
2. **Space efficiency matters more over RDMA**. The $O(S_1 \cdot P)$ space bound prevents RDMA buffer exhaustion. Under child stealing, unbounded task queues translate to unbounded memory registration, which exhausts NIC resources.
3. **Uni-address thread management** reduces virtual memory consumption, making RDMA-based work-stealing feasible at scale.

5.4 Cache Hierarchy → Memory Hierarchy → Network Hierarchy

The cache miss analysis from Section 3 extends across the full memory hierarchy:

Table 3: Work-first locality benefits across the memory hierarchy

Level	"Cache Miss" Cost	Latency	Work-First Benefit
L1 → L2	CPU cache miss	∼5 ns	Serial access pattern preserved
L2 → DRAM	LLC miss	∼100 ns	Depth-first reduces working set
DRAM → GPU	PCIe/NVLink transfer	∼1 μs	Prefetch on critical path only
GPU → remote GPU	RDMA transfer	∼1–5 μs	HOT tensors get dedicated QP

At each level, the work-first principle says the same thing: optimize the common case (data is local) and accept higher overhead on the rare case (data must be fetched). The ratio of common-to-rare improves at each level because deeper transfers are more expensive, making the optimization more valuable.

6 Implications for the Middleware

6.1 Scheduling Policy Selection

SLAW's adaptive policy selection [6] suggests the WFA classifier should not use a single fixed strategy. Different tensor types benefit from different scheduling policies:

Table 4: Tensor type to scheduling policy mapping

Tensor Type	Access Pattern	Policy	Rationale
Attention KV (induction)	Sparse, long-range	Work-first	Critical path, cache-sensitive
Attention KV (local)	Dense, sequential	Lane (static)	Predictable, NUMA-friendly
MLP activations	Batch-uniform	Lane (static)	Homogeneous across tokens
Gradient all-reduce	Bulk, periodic	Help-first	Distribute across all links
Pipeline activations	Sequential, per-stage	Work-first	Depth-first minimizes bubbles

6.2 NUMA-Aware Buffer Placement

The NUMA-aware extensions to work-stealing have a direct analog in RDMA buffer placement. On our ConnectX-4 setup, the NIC is attached to a specific NUMA node. RDMA buffers should be allocated on the NIC's local NUMA node to avoid cross-socket penalties on the critical path [7].

For the two-tower setup:

- **Tower 1** (Windows, 5070 Ti): Single-socket, no NUMA concern. TCP/gRPC fallback means buffer placement affects PCIe locality only.
- **Tower 2** (Linux, 2× V100): Multi-GPU with ConnectX-4. RDMA buffers must be on the NUMA node closest to the NIC. GPU tensors for GPUDirect RDMA must be on the GPU whose PCIe root port shares the NIC's NUMA domain.

6.3 Space-Bounded Transfer Queues

The $O(S_1 \cdot P)$ space bound under continuation stealing motivates bounded RDMA transfer queues. Rather than allowing unbounded outstanding RDMA operations (which exhaust NIC send/receive queue entries), the middleware should:

1. Maintain a fixed-depth queue per QP, sized proportionally to the pipeline depth.
2. Use back-pressure (completion polling) when the queue fills, rather than allocating more entries.
3. Prioritize draining the queue (completing in-flight transfers) over posting new operations—the work-first principle applied to the NIC itself.

7 Quantitative Analysis

7.1 Steal Frequency in Tensor Transport

From Equation 1, the number of steals is $O(P \cdot T_\infty)$. For pipeline-parallel inference with L layers distributed across P nodes:

$$T_\infty = L \cdot t_{\text{layer}} + (P - 1) \cdot t_{\text{transfer}} \tag{3}$$

With $L = 32$ layers, $P = 2$ nodes, $t_{\text{layer}} \approx 5$ ms, and $t_{\text{transfer}} \approx 0.5$ ms (12 KB activation at 100 Gbps), the transfer overhead is <3% of the critical path. The work-first principle predicts that optimizing the 97% (compute) matters far more than optimizing the 3% (transfer coordination).

However, the transfer latency is not the full cost. Each cross-node transfer incurs:

- **Registration cost**: ∼10 μs for on-demand `ibv_reg_mr`. Pre-registration (HOT path) eliminates this.
- **Cache pollution**: Incoming RDMA writes invalidate CPU cache lines. DMA-coherent buffers avoid this.
- **Synchronization**: Completion notification adds ∼1 μs (polled) to ∼10 μs (event-driven). HOT path uses polling.

The total per-transfer overhead on the fast path (HOT, pre-registered, polled) is ∼2 μs. On the slow path (COLD, on-demand registration, event-driven) it is ∼25 μs. The 12× difference justifies the WFA classifier's resource allocation: dedicating QPs and pre-registered buffers to HOT tensors is the work-first principle in action.

8 Connection to Prior Updates

Table 5: Work-first concepts mapped to existing middleware components

WFA Concept	**Middleware Component**	**Update**
Fast clone (zero overhead path)	HOT tensor: dedicated QP, polled	Update 10
Slow clone (coordination path)	COLD tensor: batched, compressed	Update 10
Adaptive policy selection (SLAW)	WFA classifier θ_1/θ_2 thresholds	Update 10
Depth-first cache locality	Prefetch FSM following access chains	Update 10
Composition-aware priority	Composition-weighted WFA	Update 11
NUMA-aware stealing	NIC-local buffer placement	Update 8
Space-bounded queues	Fixed-depth QP send queues	`transport.rs`

The WFA tensor classifier was designed empirically in Update 10 and given circuit-awareness in Update 11. This update provides the **theoretical foundation**: the hot/warm/cold classification is an instance of the work-first principle applied to a memory hierarchy that extends from CPU cache through GPU memory to RDMA network transfers. The thresholds θ_1 and θ_2 correspond to the boundary between "common case" (local, fast) and "rare case" (remote, slow) in the work-stealing framework.

9 Conclusions

The work-first principle provides a unifying framework for understanding the scheduling decisions in the RDMA middleware:

1. **CPU scheduling**: Work-stealing with continuation stealing minimizes overhead on the serial path. The WFA classifier's HOT/WARM/COLD tiers implement the same principle for tensor transport—zero overhead on the critical compute path, higher overhead acceptable for non-critical transfers.

2. **Memory scheduling**: Depth-first execution under work-first preserves cache locality. The middleware's prefetch FSM follows the same principle—prefetch along the depth-first computation path (layer-by-layer pipeline), not breadth-first across all possible future accesses.

3. **Network scheduling**: The $O(S_1 \cdot P)$ space bound motivates bounded RDMA transfer queues. NUMA-aware work-stealing maps to NIC-local buffer placement. The steal frequency bound $O(P \cdot T_\infty)$ predicts that cross-node transfers are rare relative to local computation, justifying aggressive optimization of the local (fast) path.

4. **Adaptive policy**: SLAW's per-task policy selection validates the WFA classifier's per-tensor strategy assignment. Neither work-first nor help-first universally dominates; the classifier selects based on observed access patterns, just as SLAW selects based on locality hints.

Fleury's lane model [3] provides an alternative framing for the homogeneous-work case common in LLM inference: when all tokens in a batch undergo identical computation, static SPMD partitioning with barrier synchronization is simpler and equally efficient. The middleware should use lane-style scheduling within a pipeline stage (homogeneous token processing) and work-first stealing between stages (heterogeneous pipeline scheduling).

References

[1] Umut A. Acar, Guy E. Blelloch, and Robert D. Blumofe. The data locality of work stealing. *Theory of Computing Systems*, 35(3):321–347, 2002.

[2] Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, 1999.

[3] Ryan Fleury. Multi-core by default, 2025.

[4] Matteo Frigo, Charles E. Leiserson, and Keith H. Randall. The implementation of the Cilk-5 multithreaded language. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 212–223, 1998.

[5] Yi Guo, Rajkishore Barik, Raghavan Raman, and Vivek Sarkar. Work-first and help-first scheduling policies for async-finish task parallelism. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2009.

[6] Yi Guo, Jisheng Zhao, Vincent Cave, and Vivek Sarkar. SLAW: A scalable locality-aware adaptive work-stealing scheduler. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2010.

[7] Shumpei Shiina and Kenjiro Taura. Distributed continuation stealing is more scalable than you might think. In *IEEE International Conference on Cluster Computing (Cluster)*, 2022.